



Security Assessment Final Report



Polymarket Deposit Wallet Upgrade

June 2026

Prepared for Polymarket

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Findings Summary.....	6
Severity Matrix.....	6
Detailed Findings.....	7
Medium Severity Issues.....	8
M-01: New ERC1271 functionality enables ghost fills.....	8
Informational Severity Issues.....	9
I-01: ERC-1271 failure value is a magic number instead of a named constant.....	9
I-02: isValidSignature() reverts on an expired session signer instead of returning a non-magic value...	10
Disclaimer.....	12
About Certora.....	12

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polymarket Deposit Wallet Upgrade	https://github.com/Polymarket/deposit-wallet/	Initial: c08ba3c Fix review: 500f184	EVM, Solidity

Project Overview

This document describes the security review of **Polymarket Deposit Wallet Upgrade**. The work was undertaken from **June 15th, 2026** to **June 22nd, 2026**.

The following contract list is included in our scope:

```
src/DepositWallet.sol
src/DepositWalletFactory.sol
src/libraries/SessionSignerLib.sol
src/libraries/SignatureVerifierLib.sol
src/libraries/PasskeySignerLib.sol
src/libraries/WebAuthnNative.sol
src/libraries/Ownable.sol
src/interfaces/IDepositWallet.sol
src/Errors.sol
src/Events.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The deposit wallet system is a fleet of deterministic, factory-deployed smart-contract wallets (Foundry + Solady). Users own a wallet deployed as an ERC-1967 proxy; an operator relays EIP-712-signed call batches through the factory (`factory.proxy()` → `wallet.execute()`); owners hold a pause + timelock emergency exit hatch for direct asset recovery. Wallets are upgraded collectively through a `DepositWalletBeacon` owned by a timelock.

The upgrade introduces three coupled workstreams:

1. **Unified factory** — `DepositWalletFactory` now deploys *both* proxy shapes: forward-looking ERC-1967 **beacon proxies** (`deploy`, operator role) and **legacy ERC-1967 UUPS proxies** embedding the live V1 wallet implementation (`deployLegacy`, new exclusive `ROLE_2`). The same implementation is installed on the live Polygon factory proxy via UUPS upgrade (Phase D) and deployed fresh at the canonical address on new chains for cross-chain fund recovery.
2. **Passkey session signers** — owners can authorize P-256 (secp256r1) WebAuthn passkeys as session signers; assertions are verified on-chain via the RIP-7212 precompile.
3. **ERC-1271 signers** — contract accounts (and EIP-7702-delegated EOAs) can act as wallet owners and as session signers; ECDSA-then-ERC-1271 verification is centralized in `SignatureVerifierLib` and used by `execute`, `isValidSignature`, and ownership handover.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	0
Low	-	-	-
Informational	2	2	2
Total	3	3	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	New ERC1271 functionality enables ghost fills	Medium	Acknowledged
I-01	ERC-1271 failure value is a magic number instead of a named constant	Informational	Fixed
I-02	isValidSignature() reverts on an expired session signer instead of returning a non-magic value	Informational	Fixed

Medium Severity Issues

M-01: New ERC1271 functionality enables ghost fills

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/DepositWallet.sol	Status: Acknowledged	

Description:

Upon deposit wallet operations that require a signature, there is now a functionality which allows the signer to be an ERC1271 contract. This allows ghost fills via the following way:

1. Deposit wallet has a session signer that is an ERC1271 contract.
2. `isValidSignature` returns `true`, thus he successfully gets filled off-chain.
3. The signer frontruns on-chain settlement and makes `isValidSignature` return `false`.
4. Now on-chain settlement fails, causing a ghost fill.

Recommendations:

There may not be a purely on-chain mitigation if the protocol accepts mutable ERC-1271 validity during off-chain order matching. Consider restricting ERC-1271 session signers to trusted accounts, enforcing API-level allowlisting, or requiring signatures whose validity cannot be revoked or changed before settlement.

Customer Response:

Acknowledged. We only allow such orders from trusted users, this is protected at our API level.

Informational Severity Issues

I-01: ERC-1271 failure value is a magic number instead of a named constant

Files:

src/DepositWallet.sol

Description:

`isValidSignature` returns the raw literal `0xffffffff` to signal an invalid signature, while the success path uses the named constant `ERC1271_MAGIC_VALUE` from `WalletLib.sol`.

EIP-1271 fixes only the success value (`0x1626ba7e`); any other `bytes4` means "invalid". So `0xffffffff` is just a value the contract picked to mean rejection, and as a bare literal it hides that intent. The same literal also appears at two return sites, so a future edit could change one and miss the other.

No security impact. Callers compare against `ERC1271_MAGIC_VALUE`, and any non-matching value is correctly treated as a rejection. This is readability and maintainability only.

Recommendations:

Add a named constant beside `ERC1271_MAGIC_VALUE` in `WalletLib.sol` and use it at both return sites:

```
/// @dev Returned by `isValidSignature` to signal an invalid signature.  
bytes4 constant ERC1271_INVALID_VALUE = 0xffffffff;
```

Customer Response:

Fixed in [PR#71](#)

Fix Review:

Fix confirmed

I-02: `isValidSignature()` reverts on an expired session signer instead of returning a non-magic value

Files:

src/DepositWallet.sol

Description:

`isValidSignature()` reverts when a signature comes from an expired session signer, instead of returning a non-magic value. EIP-1271's reference implementation signals an invalid signature with a return value (`return 0xffffffff`), and an expired signer is just an invalid signature for the wallet.

```
require(
    block.timestamp < sessionSignerAuthorizedUntil(signer),
    SessionSignerUnauthorized()
); //@audit reverts instead of returning the failure value
```

The P-256 branch behaves the same way. Note that this is also internally inconsistent: a malformed envelope and an unsupported signer kind both return `0xffffffff`, while an expired authorization reverts. Session expiry is a normal lifecycle event, so this path is hit in ordinary use.

Recommendations:

Return the failure value for an expired or unauthorized session signer instead of reverting, in both the secp256k1 and P-256 branches:

```
if (block.timestamp >= sessionSignerAuthorizedUntil(signer)) return
ERC1271_INVALID_VALUE;
```

(`ERC1271_INVALID_VALUE` is the named failure constant proposed in [I-1].)

Customer Response:

Fixed in [PR#71](#)

Fix Review:

Fix confirmed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.